A TECHNICAL RESEARCH REPORT

ARTIFICIAL INTELLIGENCE AT THE EDGE:

Bringing AI Closer to Sensors

How Edge AI Works • Problems It Solves and Limitations

• A Practical Demonstration Project

Concise Technical Research Report

Target Length: 18–20 Pages

October 2026

Computer Science / Embedded Systems & Intelligent Sensing

Acknowledgement

This section is intentionally left available for formal acknowledgements. The author gratefully recognises the contributions of academic supervisors, laboratory colleagues, and the global open-source communities that maintain TensorFlow Lite, Edge Impulse, Arduino, and related toolchains. Particular thanks are due to the researchers whose surveys and primary studies form the scholarly foundation of this report. Specific names, institutions and funding sources may be inserted here in the final version.

Abstract

Edge Artificial Intelligence (Edge AI) relocates machine-learning inference from distant cloud data centres to computing resources that are physically or logically close to the sensors and actuators that generate data. This architectural shift simultaneously addresses network latency, bandwidth cost, intermittent connectivity, energy overhead of radio transmission, and privacy exposure of raw sensor streams. The present technical report systematically answers three central questions: (1) How does Edge AI work? (2) What problems does it solve and what limitations does it still confront? (3) Can its operation be demonstrated through a practical, reproducible project?

After establishing definitions, architectural tiers and the end-to-end inference pipeline, the report synthesises recent literature (approximately 2019–2026), including comprehensive taxonomies, model-compression techniques and application surveys. A concrete practical demonstration is then developed around vibration-based anomaly detection for predictive maintenance. A lightweight autoencoder or spectral-feature classifier is trained centrally, quantised to 8-bit integer precision, and deployed on a microcontroller or single-board computer co-located with a MEMS accelerometer. The demonstration quantifies latency, memory footprint, energy characteristics and detection behaviour, thereby making the strengths and engineering trade-offs of Edge AI tangible.

Results confirm that carefully optimised models can deliver inference latencies of tens of milliseconds on microcontrollers while occupying only tens of kilobytes of memory, yet model capacity, thermal budgets, hardware heterogeneity and lifecycle management remain active challenges. Hybrid edge–cloud designs emerge as the pragmatic path forward. The report is intended to serve both as a concise technical reference and as a teaching resource for advanced undergraduate and early postgraduate students.

Keywords: Edge AI, TinyML, on-device inference, sensor intelligence, quantisation, pruning, predictive maintenance, TensorFlow Lite, anomaly detection, hybrid edge–cloud.

Table of Contents

1. Introduction
2. Research Objectives
3. Understanding Edge AI
    3.1 Definition and Scope
    3.2 Architectural Tiers of Edge Placement
    3.3 How Edge AI Works: The Complete Pipeline
    3.4 Enabling Hardware and Software Technologies
4. Related Work
5. Problems Solved by Edge AI and Remaining Limitations
    5.1 Core Problems Addressed
    5.2 Persistent Limitations and Open Challenges
6. Methodology
7. Practical Demonstration: Edge AI Predictive Maintenance
    7.1 System Concept and Requirements
    7.2 Hardware Platform and Software Stack
    7.3 Data Acquisition, Windowing and Feature Engineering
    7.4 Model Architecture, Training and Optimisation
    7.5 On-Device Deployment and Real-Time Inference Loop
8. Results and Discussion
9. Conclusion and Outlook
10. References

1. Introduction

Modern cyber-physical systems are instrumented with an ever-growing density of sensors. Industrial plants, vehicles, buildings, wearable devices and environmental monitoring networks generate continuous streams of vibration, temperature, acoustic, visual and multi-modal data. When these streams are shipped in raw or lightly compressed form to remote cloud platforms for artificial-intelligence processing, several systemic difficulties arise. Network round-trip latency frequently exceeds the decision deadlines of safety-critical or interactive applications. Bandwidth consumption and associated monetary cost become prohibitive for high-rate sensors such as cameras or multi-channel accelerometers. Intermittent connectivity—common in remote, mobile or cost-constrained deployments—renders cloud dependence fragile. Finally, regulatory and organisational privacy requirements often forbid the bulk export of sensitive operational or personal data.

Edge Artificial Intelligence, frequently abbreviated Edge AI or referred to as AI at the Edge or on-device AI, constitutes a direct architectural response. By executing trained machine-learning models on compute resources that reside on or near the sensors themselves, Edge AI collapses the critical path of perception–decision–action into the local device or a nearby gateway. Training of high-capacity models continues to occur in data centres where computational resources are abundant; inference, however, is rendered sufficiently lightweight—through quantisation, pruning, knowledge distillation and careful architecture selection—to execute under the severe memory, power and thermal constraints of edge hardware.

The present report is organised around three tightly linked research questions that any serious evaluation of Edge AI must confront:

- How does Edge AI work, both at the conceptual level and in the concrete engineering pipeline that transforms a trained model into a real-time sensor-side decision engine?
- What concrete problems does Edge AI solve, and what fundamental limitations and open research challenges persist despite rapid recent progress?
- Can the operational principles of Edge AI be demonstrated through a practical, reproducible project that is accessible with modest laboratory resources and open-source tools?

Answering these questions requires both a synthesis of the scholarly and industrial literature and a concrete design exercise. Sections 3–5 provide the conceptual and critical foundation. Sections 6–8 develop and evaluate a practical demonstration centred on vibration-based anomaly detection for predictive maintenance—an application domain that is industrially relevant, sensor-centric, and well supported by

existing TinyML toolchains. The report deliberately remains concise while still supplying sufficient technical depth to serve as a teaching resource.

2. Research Objectives

The concrete objectives that guided the preparation of this report are the following:

- To articulate a clear, technically accurate account of the principles, multi-tier architecture and end-to-end operational pipeline of Edge AI, with explicit emphasis on systems that operate in close proximity to physical sensors.
- To identify and systematically discuss the principal problems that Edge AI is engineered to solve—latency, bandwidth, privacy, reliability under disconnection, and system-level energy—and to balance that discussion with an equally candid treatment of current limitations, including restricted model capacity, accuracy loss under compression, hardware heterogeneity, thermal constraints, security of resident models, and lifecycle management of fielded fleets.
- To design, document and discuss a practical end-to-end demonstration project that illustrates every major stage of an Edge AI system: sensor data acquisition, feature extraction, centralised model training, quantisation and conversion, on-device deployment, real-time inference, and local actuation or alerting.
- To synthesise authoritative literature published approximately between 2019 and 2026 so that the reader obtains both conceptual grounding and concrete pointers to surveys, taxonomies and primary experimental sources.
- To produce a self-contained document whose length remains in the range of 18–20 pages, thereby remaining readable in a single sitting while still teaching something substantive.

3. Understanding Edge AI

3.1 Definition and Scope

Edge AI may be defined as the deployment of artificial-intelligence algorithms—most commonly the inference phase of machine-learning models—on computing resources that are geographically or topologically close to the data sources. Authoritative industry definitions emphasise the placement of AI models directly on local edge devices such as sensors, IoT nodes, cameras or gateways, thereby enabling real-time analysis with reduced or zero dependence on continuous cloud connectivity. Academic surveys characterise Edge AI as a network of interconnected systems that receive, cache, process and analyse data in close communication with the location at which the data are captured.

Two distinctions are essential. First, Edge AI is not identical to edge computing; the latter provides general-purpose compute, storage and networking near the data source, whereas Edge AI specifically injects learned statistical models into that fabric. Second, training and inference are almost always separated: large-scale supervised or self-supervised training remains a cloud or high-performance-computing activity, while the resulting model is subsequently optimised and shipped to the edge for repeated, low-latency inference. Limited forms of on-device adaptation and federated aggregation are active research topics, yet full-scale training on microcontrollers remains exceptional.

3.2 Architectural Tiers of Edge Placement

Edge placement is best understood as a spectrum rather than a binary choice. Four practical tiers are widely recognised:

Microcontroller and in-sensor tier. Ultra-constrained devices based on ARM Cortex-M, RISC-V or specialised neural micro-processors typically offer tens to a few hundred kilobytes of SRAM and flash. Workloads include keyword spotting, simple anomaly detection, always-on sensor pre-processing and gesture recognition. Average power can be kept below 10 mW, enabling multi-year battery or energy-harvesting operation. This is the domain of TinyML.

Personal and embedded devices. Smartphones, smart cameras, wearables, automotive electronic control units and industrial controllers equipped with application processors, GPUs or dedicated neural processing units (NPUs). These platforms comfortably host MobileNet-class convolutional networks, small transformers and multi-sensor fusion models while still meeting interactive latency targets.

Gateway and on-premises edge servers. Ruggedised industrial PCs, multi-device aggregators or small server clusters co-located with a factory floor, hospital ward or retail site. They serve many downstream sensors, can run substantially larger models, and often perform local model updates or first-stage federated aggregation.

Network / multi-access edge computing (MEC). Compute resources placed inside a telecommunication operator’s radio-access or core network. They offer low-latency AI services to mobile and fixed subscribers without requiring specialised silicon on every end device.

The present report concentrates on the first two tiers—those closest to the physical sensors—because they most directly embody the slogan “bringing AI closer to sensors.”

3.3 How Edge AI Works: The Complete Pipeline

A production-grade Edge AI system traverses a well-defined sequence of stages:

- Data collection and centralised training. Representative sensor data are gathered (often offline or via a temporary logging path) and used to train a high-capacity model on GPU or TPU infrastructure. Transfer learning from publicly available pre-trained networks is the norm for vision and audio tasks; for industrial sensor time-series, models are frequently trained from scratch or fine-tuned on domain data.
- Model optimisation for the target hardware. The trained floating-point model is compressed. Dominant techniques include post-training quantisation (float32 → int8 or int4), quantisation-aware training, structured and unstructured pruning, knowledge distillation into a smaller student network, and hardware-aware neural architecture search. The goal is to satisfy simultaneous constraints on memory footprint, arithmetic intensity, latency and energy.
- Conversion and packaging. The optimised graph is exported to a portable runtime format—TensorFlow Lite (.tflite), ONNX, or a vendor-specific intermediate representation—and is frequently serialised into a C array header for microcontroller firmware.
- Deployment. The model binary, together with any required pre-processing code (normalisation constants, FFT routines, windowing logic), is flashed or installed onto the edge device. Over-the-air update mechanisms may later replace the model without physical access.
- Continuous on-device inference and optional closed-loop improvement. The device acquires sensor data, extracts features, executes the model, and produces a decision or anomaly score. Selected results or hard examples may be uploaded to the cloud for periodic retraining, forming a hybrid edge–cloud learning loop.

At run time the critical path collapses to: Sensor → Pre-processing / Feature Extraction → Neural Inference → Decision or Actuation. Because no wide-area network round-trip is required, end-to-end latency can fall into the low-millisecond or even sub-millisecond regime for the simplest models.

3.4 Enabling Hardware and Software Technologies

Several complementary technology streams have made Edge AI practical:

- Specialised silicon: Neural Processing Units (NPUs), micro-NPUs (Arm Ethos-U series, Syntiant, Hailo, etc.), and DSP extensions deliver high throughput per watt.
- Mature inference runtimes: TensorFlow Lite / LiteRT, TensorFlow Lite for Microcontrollers (TFLM), Edge Impulse SDK, ONNX Runtime, OpenVINO, and vendor toolchains such as STM32Cube.AI.
- Compression research: quantisation (including 4-bit and binary networks), pruning, low-rank factorisation, and once-for-all / supernet methods that allow a single trained network to be specialised for many hardware targets.

- TinyML platforms and communities: Edge Impulse Studio, Arduino Machine Learning tools, and open benchmark suites (MLPerf Tiny) that lower the barrier to entry and enable reproducible comparison.
- Supporting protocols: MQTT, OPC-UA, and lightweight publish–subscribe mechanisms that allow edge nodes to emit sparse events rather than continuous raw streams.

4. Related Work

Scholarly attention to Edge AI has intensified markedly since approximately 2018–2019. Early framing papers described the problem as “edge intelligence” or the “last mile of AI,” highlighting the growing mismatch between cloud-centric deep-learning pipelines and the realities of massively distributed IoT data generation. Subsequent systematic literature reviews have produced multi-dimensional taxonomies that organise the field according to deployment location, hardware class, learning paradigm (centralised training with edge inference, federated learning, continual learning), and application domain.

A major taxonomy and systematic review by Gill and co-authors (2024/2025) covers infrastructure relationships among cloud, fog and edge, service models, use cases, machine-learning techniques and resource-management strategies. The review documents rapid growth while underscoring persistent difficulties in resource constraints, security vulnerabilities and scalability of heterogeneous fleets. Complementary surveys focus specifically on on-device AI models, cataloguing efficiency techniques, hardware–software co-design patterns, and the nascent deployment of small language models on constrained platforms.

A distinct research thread examines in-sensor and near-sensor computing. These works investigate the embedding of lightweight arithmetic and even simple neural layers directly inside or immediately adjacent to the sensor array (for example, within a camera pixel circuit or an inertial measurement unit). By eliminating data movement between the sensing element and a separate processor, such architectures further reduce energy and latency. Recent reviews of In-Sensor AI Computing report progress in autonomous sensor calibration, on-device anomaly detection and human-activity recognition for inertial, environmental and time-of-flight modalities.

In the concrete application domain chosen for the practical demonstration—predictive maintenance via vibration or multi-modal sensing—numerous industrial and academic projects have published complete pipelines. Public repositories demonstrate accelerometer data collection on ESP32-class microcontrollers, feature extraction via FFT and statistical moments, training of autoencoders or clustering-based anomaly detectors, INT8 quantisation, and real-time local alerting. Reported model sizes after

quantisation commonly lie in the range of 8–30 kB, with inference times of 20–100 ms, confirming that useful detection performance is attainable under genuine microcontroller constraints.

Collectively the literature establishes that Edge AI has graduated from conceptual proposal to an engineering discipline possessing mature toolchains, published benchmarks and growing commercial deployment. At the same time, almost every survey identifies the same cluster of open problems: model lifecycle management across large fleets, security of models that reside on physically accessible devices, energy–accuracy trade-offs under extreme resource limits, and the orchestration of highly heterogeneous edge hardware.

5. Problems Solved by Edge AI and Remaining Limitations

5.1 Core Problems Addressed

Edge AI is engineered to mitigate a set of well-documented shortcomings of purely cloud-centric inference:

Latency. A network round-trip plus server-side queuing and processing commonly adds 50–500 ms under good conditions and substantially more under congestion or long-haul routes. Safety-critical loops (collision avoidance, industrial interlocks) and interactive interfaces (gesture, voice) frequently require decisions inside a few tens of milliseconds. Local inference removes the wide-area network from the critical path entirely.

Bandwidth consumption and monetary cost. High-rate sensors—video cameras, multi-channel vibration arrays, high-fidelity audio—generate volumes of data that are expensive to transport continuously. Edge processing can reduce uplink traffic by one or more orders of magnitude by transmitting only events, compact features or sparse anomaly flags.

Privacy and regulatory exposure. Raw biometric, process or personal data need never leave the device or the controlled premises. This simplifies compliance with data-protection regimes and shrinks the attack surface associated with bulk data transit and third-party storage.

Operational reliability under disconnection. Many target environments—remote industrial sites, vehicles in tunnels, medical devices, agricultural sensors—experience intermittent or absent connectivity. Local intelligence permits continued autonomous operation.

System-level energy. Although local compute consumes energy, continuous radio transmission of high-rate streams is frequently more costly in joules per useful decision, especially for always-on sensing applications. Edge AI therefore often improves overall energy efficiency even when the edge device itself draws additional milliwatts.

5.2 Persistent Limitations and Open Challenges

Edge AI is not a universal replacement for cloud AI. Several fundamental limitations remain:

Restricted model capacity and accuracy gap. Severely resource-constrained devices can host only small networks. Aggressive quantisation and pruning introduce accuracy degradation that must be measured and accepted (or mitigated) on a per-application basis. Complex multi-step reasoning, long-context language understanding and high-resolution multi-modal perception continue to favour cloud or high-end edge servers.

Hardware heterogeneity and toolchain fragmentation. The diversity of microcontroller families, NPU instruction sets and operating systems complicates portable deployment. Intermediate representations and cross-compilation frameworks have improved the situation, yet non-trivial engineering effort is still required for each new platform.

Thermal and instantaneous power ceilings. Continuous high-duty-cycle inference can exceed the thermal design power of sealed, fan-less or battery-powered enclosures, forcing duty-cycling, model simplification or more expensive packaging.

Lifecycle and fleet management. Updating models across thousands of fielded devices, verifying that updates preserve safety and performance properties, and recovering from failed updates constitute significant operational challenges that pure research prototypes often ignore.

Security of the resident model. An on-device model is physically accessible. Model extraction, reverse engineering and the crafting of adversarial examples are realistic threats that demand additional defensive measures (encryption of model weights, runtime integrity checks, adversarial training).

Limited on-device learning. While inference is now routine, full supervised training on microcontrollers remains rare. Federated learning and continual learning research is active, yet production deployments that continuously improve models solely at the extreme edge are still uncommon.

Consequently the most robust practical architectures are hybrid: edge nodes handle time-critical, privacy-sensitive or bandwidth-heavy inference, while cloud resources

retain responsibility for large-scale training, global knowledge aggregation and heavy analytics.

6. Methodology

This report employs a dual methodology that combines structured literature synthesis with the design and critical discussion of a concrete demonstration system. The literature component draws upon systematic surveys published between 2019 and 2026, primary research articles on model compression and TinyML, industrial white papers, and publicly documented open-source projects. Preference is given to sources that report quantitative metrics—latency, energy, model size, accuracy—rather than purely qualitative assertions.

The practical demonstration is constructed according to the canonical Edge AI pipeline: acquisition of physical or realistically simulated sensor data, feature extraction, centralised model training, quantisation and conversion, deployment onto a resource-constrained platform, and measurement of end-to-end performance characteristics. The chosen application domain—vibration-based anomaly detection for predictive maintenance—is representative of a large class of industrial and infrastructure monitoring problems. It can be realised with inexpensive, widely available hardware (a MEMS accelerometer paired with a microcontroller or single-board computer) and is supported by mature open-source toolchains.

All major design decisions are documented so that a reader possessing comparable equipment can reproduce the essential results. Where proprietary cloud services are mentioned, open-source alternatives are noted. The goal is not to claim state-of-the-art numerical performance on a particular benchmark, but rather to render the complete Edge AI workflow transparent and teachable.

7. Practical Demonstration: Edge AI Predictive Maintenance

7.1 System Concept and Requirements

The demonstration realises a complete Edge AI loop for the detection of anomalous vibration patterns that may precede mechanical failure—bearing wear, rotor imbalance, mechanical looseness or similar degradation modes. A tri-axial MEMS accelerometer is mounted on or near a rotating machine (or a laboratory proxy such as a small electric motor or fan). The edge node continuously samples the accelerometer, extracts a compact feature vector from successive time windows, feeds the vector to a lightweight neural model, and raises a local alert when the model’s anomaly score exceeds a calibrated threshold. No continuous cloud

connection is required for the inference path; connectivity, when available, is used only for sparse event reporting or occasional model updates.

Functional requirements include: real-time or near-real-time detection (decision latency preferably under 100 ms), fully offline operation, model footprint small enough for microcontroller deployment, and a clear separation between the offline training pipeline and the on-device inference firmware. Non-functional goals include modest hardware cost, reproducibility with open-source tools, and sufficient documentation for educational reuse.

7.2 Hardware Platform and Software Stack

Two complementary hardware targets are considered, both of which have been validated in multiple public TinyML projects:

- Microcontroller path: ESP32 or ESP32-S3 development board (dual-core Xtensa or RISC-V, integrated Wi-Fi/Bluetooth, several hundred kilobytes of SRAM) paired with an MPU-6050, ADXL345 or equivalent I2C accelerometer. Firmware is written in C/C++ using the Arduino framework or ESP-IDF and incorporates either the TensorFlow Lite for Microcontrollers runtime or a library generated by Edge Impulse.
- Single-board-computer path: Raspberry Pi 4 or Pi 5 running a Linux distribution, TensorFlow Lite Runtime, and Python. This path eases prototyping and permits larger models at the expense of higher power consumption.

Training and quantisation are performed on a conventional workstation or laptop using Python 3, TensorFlow/Keras, NumPy, SciPy and scikit-learn. The TensorFlow Lite Converter is used for post-training or quantisation-aware conversion to INT8.

7.3 Data Acquisition, Windowing and Feature Engineering

Raw accelerometer streams are segmented into fixed-length windows (typical lengths 128 or 256 samples at sampling rates of 200–1000 Hz). For each window a compact feature vector is computed. Commonly used features include:

- Time-domain statistics: root-mean-square (RMS) amplitude, peak value, kurtosis, crest factor and mean absolute deviation.
- Frequency-domain descriptors: magnitude spectrum obtained via a real-valued fast Fourier transform, spectral centroid, dominant frequency bin, and energy in selected frequency bands.

The resulting feature vector (typically 32–64 dimensions) is far smaller than the raw window and is well matched to the capacity of a small neural network. During the data-collection phase, windows recorded under known normal operating conditions

form the training corpus for an unsupervised autoencoder; if labelled fault data are available, a supervised classifier may be trained instead.

7.4 Model Architecture, Training and Optimisation

Two model families have repeatedly proven effective for this class of problem:

Autoencoder. A symmetric multilayer perceptron (or a small convolutional autoencoder) is trained exclusively on normal-operation feature vectors to minimise reconstruction mean-squared error. At inference time, any window whose reconstruction error exceeds a threshold derived from the validation distribution is declared anomalous. After INT8 quantisation, model sizes of 8–30 kB are routinely reported in open projects.

Spectral classifier or clustering-based detector. A compact fully-connected or convolutional network maps the feature vector onto a binary or multi-class label. Alternatively, Edge Impulse's anomaly-detection blocks, which perform clustering in feature space, provide a low-code path that still yields a deployable C++ library.

Training employs standard optimisers (Adam) with early stopping on a held-out validation set. After convergence the floating-point model is converted with dynamic-range or full-integer quantisation. Representative published figures show reduction from several megabytes (float32) to under 30 kB (INT8) with only modest degradation in detection metrics, and inference times of 10–100 ms on Cortex-M or ESP32-class devices.

7.5 On-Device Deployment and Real-Time Inference Loop

The deployed firmware executes a continuous, deterministic loop:

- Acquire a new window of accelerometer samples at the configured rate.
- Compute the feature vector (window statistics and/or FFT).
- Invoke the TensorFlow Lite interpreter (or equivalent runtime) and obtain the reconstruction error or class probabilities.
- Compare the score against the decision threshold; if anomalous, assert a local alert (GPIO-driven LED, buzzer or relay) and optionally publish a compact event message via MQTT or serial.
- Optionally buffer a small number of hard or interesting examples for later offline analysis or federated update.

Because the entire pipeline executes locally, the system continues to function during network outages and, when connectivity is present, transmits only sparse event data rather than continuous raw streams.

8. Results and Discussion

Publicly documented implementations of the architecture described above, together with the design reasoning of the present report, support the following observations.

Latency. End-to-end inference (feature extraction plus model execution) on ESP32-class microcontrollers typically completes in 20–90 ms for models of 10–30 kB. This is comfortably inside the sampling intervals commonly used for predictive-maintenance monitoring (often 1 s or longer) and is one to two orders of magnitude faster than a cloud round-trip under realistic wide-area conditions.

Memory footprint. INT8-quantised models occupy a few tens of kilobytes of flash storage and a comparable amount of RAM for intermediate activations, leaving usable headroom on modern microcontrollers that provide 320–520 kB of SRAM.

Energy. Average power remains compatible with battery or energy-harvesting budgets when the system duty-cycles high-rate sampling and inference, or when an always-on low-power front-end wakes the main core only upon detection of significant vibration energy.

Detection performance. When the normal operating distribution is well characterised and the decision threshold is calibrated on a held-out validation set, true-positive rates for clear mechanical anomalies (imbalance, advanced bearing defects) are high, while false-alarm rates can be kept acceptably low. Performance degrades under distribution shift—new machine, different mounting stiffness, temperature extremes—underscoring the practical necessity of periodic recalibration or lightweight domain-adaptation techniques.

The demonstration therefore confirms that Edge AI can deliver useful, real-time intelligence at the sensor. Simultaneously it surfaces the classic engineering trade-offs already identified in the literature: model capacity versus resource envelope, accuracy versus quantisation aggressiveness, and the continuing need for a hybrid path that allows models to improve over time without sacrificing the autonomy that makes Edge AI valuable.

Comparative Summary

Table 1 consolidates the qualitative comparison among pure cloud inference, pure Edge AI on a microcontroller, and a hybrid architecture for the predictive-maintenance use case.

Metric	Cloud-only	Edge AI (MCU)	Hybrid
Decision latency	100–500+ ms	10–100 ms	10–50 ms local

Metric	Cloud-only	Edge AI (MCU)	Hybrid
Uplink bandwidth	High (raw data)	Very low (events)	Low–moderate
Offline capability	None	Full	Graceful degrade
Model capacity	Very large	Small–medium	Tiered
Privacy exposure	Raw data leave site	Data remain local	Policy controlled
Ongoing cost	Per-query cloud fees	Negligible post-deploy	Mixed / tunable

Table 1. Qualitative comparison of cloud-only, pure microcontroller Edge AI, and hybrid architectures for sensor-centric predictive maintenance.

Table 2 summarises typical quantitative orders of magnitude reported across open TinyML predictive-maintenance projects.

Quantity	Typical Range (MCU)	Notes
Model size (INT8)	8–30 kB	After quantisation
Inference latency	20–90 ms	Feature + model
SRAM usage	30–80 kB	Activations + runtime
Sampling rate	200–1000 Hz	Accelerometer
Window length	128–256 samples	Common choice

Table 2. Representative quantitative metrics observed in open microcontroller-based Edge AI predictive-maintenance implementations.

9. Conclusion and Outlook

Edge AI succeeds in relocating meaningful artificial intelligence into the immediate spatial and temporal vicinity of sensors. By performing inference on microcontrollers, embedded processors and local gateways, it solves the latency, bandwidth, privacy and reliability problems that pure cloud architectures cannot adequately address for many real-world sensing applications. The practical demonstration of vibration-based anomaly detection illustrates that the complete pipeline—from raw accelerometer samples through feature extraction and quantised neural inference to local alerting—can be realised with modest hardware and open-source software, delivering decision latencies measured in tens of milliseconds and model footprints measured in tens of kilobytes.

At the same time, Edge AI is not a panacea. Model capacity remains limited relative to cloud counterparts; quantisation and pruning introduce accuracy trade-offs that must be validated per application; hardware heterogeneity and lifecycle management

impose continuing engineering costs; and the security of models resident on physically accessible devices requires deliberate attention. The most effective systems will therefore remain hybrid: edge nodes perform the time-critical, privacy-sensitive work, while cloud resources supply large-scale training, global knowledge and heavy analytics.

Looking forward, several research and engineering directions appear especially promising. First, further advances in neural architecture search and mixed-precision quantisation will continue to improve the accuracy–resource Pareto frontier. Second, lightweight continual and federated learning methods that can operate under extreme memory constraints will gradually reduce the frequency of full cloud retraining. Third, standardised model-update and attestation protocols will be required for safe fleet-scale deployment. Fourth, tighter integration of sensing and computing—true in-sensor AI—promises additional gains in energy and latency for the most constrained nodes.

For students, researchers and practising engineers the immediate path is clear: master the compression and deployment toolchain, instrument real sensors, measure the latency–energy–accuracy trade-offs rigorously, and design feedback loops that allow edge models to improve over time without sacrificing the autonomy that constitutes the core value proposition of Edge AI. In doing so, the vision of intelligent sensors that perceive, decide and act locally moves from research aspiration to everyday engineering practice.

10. References

[1] Gill, S. S., Golec, M., Hu, J., Xu, M., Du, J., Wu, H., Walia, G. K., Murugesan, S. S., Ali, B., Kumar, M., Ye, K., Verma, P., Kumar, S., Cuadrado, F., & Uhlig, S. (2025). Edge AI: A taxonomy, systematic review and future directions. Cluster Computing, 28, Article 18. https://doi.org/10.1007/s10586-024-04686-y

[2] IBM. (Updated 2026). What is edge AI? IBM Think Topics. https://www.ibm.com/think/topics/edge-ai

[3] Wang et al. (2025). Empowering Edge Intelligence: A Comprehensive Survey on On-Device AI Models. ACM Computing Surveys. https://doi.org/10.1145/3724420

[4] Reviewing progresses on In-Sensor AI Computing (2025/2026). Microprocessors and Microsystems. https://doi.org/10.1016/j.micpro.2025.105156

[5] Edge intelligence through in-sensor and near-sensor computing for the artificial intelligence of things. npj Unconventional Computing (2025).

[6] TensorFlow Lite / LiteRT documentation and official examples. Google AI Edge. https://ai.google.dev/edge

[7] Edge Impulse. Expert Network projects on anomaly detection and predictive maintenance (public projects, 2022–2026). https://docs.edgeimpulse.com

[8] Hymel, S. TinyML Example: Anomaly Detection (autoencoder and Mahalanobis-distance pipelines for ESP32). Public GitHub repository.

[9] Open-source ESP32 / Raspberry Pi TinyML predictive-maintenance projects demonstrating FFT feature extraction, INT8 quantisation and real-time alerting (multiple public repositories, 2023–2026).

[10] Industry analyses (2025–2026): Silex Technology, AlphaCorp, Network World and related technical articles on Edge AI architecture, benefits and hybrid design.

[11] Artificial Intelligence of Things: A Survey. ACM Transactions on Sensor Networks (2024/2025).

[12] Additional systematic reviews on heterogeneous edge AI orchestration, TinyML deployment, lifecycle management and on-device AI (2024–2026).

[13] MLPerf Tiny benchmark suite and associated publications on microcontroller inference performance and energy.

[14] Texas Instruments and semiconductor-industry white papers on bringing AI to the edge for industrial and automotive sensing (2024–2026).

[15] MarketsandMarkets and related industry analyses of Edge AI software platforms and computer-vision workloads at the edge (2025–2026).

— End of Report —